

What is the expected interface?

A challenge in collecting SWE tasks is verifying solutions robustly without over-constraining the implementation. Over-specification (e.g., enforcing internal function names or specific helper methods) compromises the generalizability and leaks the intended solution to the model.

Expected Interface

The interface should represent the natural entry point a user or dependent system would interact with. It must never dictate internal architecture, modularization, or helper logic. The expected interface should:

1. Define minimal surface area: Specify only the primary function, the main class and its required public methods, the exact CLI command, or the API endpoint route.
2. Contain Strict Typing: Explicitly define input arguments, data types, and the expected return types or output formats. This ensures the evaluation scripts do not fail due to trivial type mismatches (e.g., returning a tuple instead of a list).
3. Be Implementation Agnostic: Exclude any mention of file structures (unless the prompt inherently requires it, like a data processing pipeline), internal state variables, or helper functions.

Evaluation Scripts

Evaluation scripts (test cases) are purely objective, programmatic assertions. They treat the submitted solution as a black-box, interacting with it solely through the Expected Interface. Scripts must not attempt to mock or patch internal components of the solution. If an evaluation requires testing internal logic directly, the Expected Interface is likely poorly designed, or the requirement should be verified using rubrics.

Test cases primarily verify:

1. Input/Output: Focus on asserting that a given input produces the exact expected output. This includes standard use cases, edge cases, and invalid inputs.
2. Side-Effects: For tasks involving databases, file systems, or network requests, tests should verify the resulting state of the system (e.g., querying a mock database to ensure a record was inserted, or checking that a file was created as required).

(Agentic) Rubrics

Rubrics serve as a qualitative and structural verification as not all constraints can be evaluated programmatically. We use atomic rubrics to verify non-deterministic outcomes, architectural choices, and qualitative constraints. Unlike evaluation scripts, they are verified with access to the codebase (i.e., white-box evaluation). Rubrics are used for:

1. Constraint Adherence: Use rubrics to verify that a solver did not use banned libraries (e.g., "The solution implemented linear regression from scratch using numpy without importing sklearn or scipy").
2. Architectural Checks: Verify design patterns and security practices that are difficult to assert programmatically (e.g., "The API correctly hashes the password using a secure algorithm like bcrypt before database insertion," or "The React component manages state using Hooks rather than Class components").
3. Qualitative Assessment: Evaluate subjective requirements, such as code readability, proper documentation, or UI/UX styling instructions.

Examples

Domain	Expected Interface	Test Case Example	Rubric Example
Algorithms	<pre>def find_shortest_path(g rid: list[list[int]]) -> int:</pre>	Assert that <code>find_shortest_path(grid_1)</code> equals 14 and handles unsolvable grids by returning -1.	The implementation utilize an optimal graph traversal algorithm like A* or BFS instead of an exhaustive brute-force search.
Data Science	<pre>def train_and_predict(tra in_csv: str, test_csv: str) -> list[float]:</pre>	Pass hidden CSV paths; assert the output list length matches test rows and RMSE is < 1.5.	The code performs feature scaling (e.g., normalization/standardization) on continuous variables prior to model training.
Web Backend	<pre>POST /api/checkout accepting JSON with user_id and cart_items.</pre>	Send a valid POST request, assert a 200 OK response, and query the mock database to ensure the order table is updated.	The endpoint wraps the database insertion and payment processing steps in a single, atomic database transaction.

Frontend/UI	A component named <code>UserProfile</code> , accepting a <code>user</code> prop object.	Render the component in a test DOM and assert the presence of specific text fields (name, email).	The layout utilizes CSS flexbox or grid to adapt to mobile screen sizes.
-------------	---	---	--